Claude Code Mods & Plugin Architecture: Verification Guide and Local Implementation

Researched 2026-10-12.

Architecture Overview: What is a Claude Mod?

According to the Claude Code Plugin Documentation (https://code.claude.com/docs/en/plugins), a **plugin** is a structured directory (defined by a `.claude-plugin/plugin.json` manifest) that packages skills (`SKILL.md`), agents, hooks, and Model Context Protocol (MCP) servers.

- **Official Definition of a 'Mod':** Anthropic specifically documents that "*a plugin that has [a hooks module written as JavaScript functions, which can also draw panes and add commands] is called a mod.*"
- **Execution Context:** Unlike standard text-based skills, a mod's hooks module runs code alongside the session, enabling interface updates (such as terminal panes and prompt-adjacent strips) and behavioral modifications.

Official Documentation vs. Reel Claims

Feature / Claim	Official Source (Claude Code Docs (https://code.claude.com/docs/en/plugins))	AI LABS Video Claim	Status
Mod Definition	A plugin with a JavaScript/TypeScript hooks module that draws panes and adds commands.	Custom plugins running inside Claude Code altering UI and logic.	**Verified**
Management UI	Managed via the `/plugin` terminal command or shell (`claude plugin disable`).	Built and enabled directly through prompt turns with automatic hot-reloading.	**Partially Verified** (Core commands verified; automated hot-reload workflow relies on creator prompt)
Local Development	Tested locally without marketplaces using `--plugin-dir <path>`.	Loaded into a temporary session cache, then permanently installed on request.	**Verified** via `--plugin-dir`
Context Overhead	Enabled plugins inject names/descriptions into context on *every* turn, consuming tokens even when unused.	Described as 'always running' like a native feature.	**Verified** (Token cost explicitly noted by Anthropic)

Feature / Claim	Official Source (Claude Code Docs (https://code.claude.com/docs/en/plugins))	AI LABS Video Claim	Status
Built-in Mod: 'You Should Know'	Not referenced in the primary plugin overview documentation.	Claimed built-in mod enabled via `/plugin enable cc-plugin-you-should-know@builtin`.	**Unverified**
Marketplace: claudemod.com	Anthropic supports catalog marketplaces (`.claude-plugin/marketplace.json`) under official, community, or third-party tiers.	Claimed community hub with hundreds of user mods.	**Unverified Third-Party**
Sponsor Discount (Runway)	N/A	Spoken: `create 15` (15% off); Description: `MKTG50` (50% off).	**Discrepant / Unreliable**

Realistic Limits & Common Pitfalls

1. **Unsandboxed Execution Security:** What a plugin executes, it runs with your local user permissions. Third-party mods or community registries (e.g., `claudemod.com` or private communities) can execute arbitrary commands or access the local filesystem. Source inspection is essential before enabling any hook module.
2. **Context Window Degradation:** Every enabled plugin places descriptions and invocation triggers into Claude's context window on every turn. Running multiple always-on mods simultaneously reduces effective context length and inflates inference costs.
3. **Interface Limitations:** Mod UI panes and status strips are terminal-native. IDE integrations (such as JetBrains or standard VS Code extensions) have distinct plugin ecosystems and do not support arbitrary terminal UI redraws.
4. **Tool Clashing:** Combining multiple mods that intercept input or rewrite responses (such as prompt enhancers, assumption checkers, and plain-language summarizers) can cause conflicting lifecycle behaviors or recursive rewrites.

Verified Commands vs. Unverified Claims

Verified Commands (from Official Documentation (https://code.claude.com/docs/en/plugins))

- Open the interactive plugin management UI:

  ```
  /plugin
  ```

- Add a documented plugin catalog repository:

```
/plugin marketplace add <marketplace-name>
```

- Disable an enabled plugin from the system shell:

```
claude plugin disable <plugin-name>
```

- Test a local plugin directory during development:

```
claude --plugin-dir ./path/to/my-plugin
```

Unverified Reel Commands (Treat as Untrusted)

- *Unverified built-in enablement:* `/plugin enable cc-plugin-you-should-know@builtin`
- *Unverified mod-specific UI triggers:* `/idea`, `/sessions`, pressing `0` with an empty prompt bar, or `Ctrl+X` followed by `Tab`.

Reusable Claude Code Authoring Prompt

To build a verified-compatible local mod without pulling untrusted third-party code, use this prompt inside an active Claude Code session:

```
Create a minimal Claude Code plugin in a local directory named 'local-task-tracker'.
1. Generate a valid .claude-plugin/plugin.json manifest containing the plugin name, version (0.1.0), and description.
2. Implement a hooks module in JavaScript that hooks into lifecycle events to print a single concise status line summarizing active work.
3. Ensure the hook runs locally without external network requests or third-party package dependencies.
4. Show the complete code and explain how to test it locally using the --plugin-dir command before any installation.
```

Original sources

- https://youtu.be/ERNqRK6sMJY